

UDYKING FIS

Financial Information System for Small and Medium Enterprises

An Automated, Direct-Access (Random Indexing) Financial
Information System for Independent Retail Fuel Stations — A Case
Study of Udyking Filling Station

Final Year Project — Defence Preparation Guide

Project Title: Design and Implementation of a Direct-Access Financial Information System for SMEs
(Udyking Filling Station)

Author: [Your Full Name]

Department: Computer Science

Supervisor: [Supervisor's Name]

Date: [Defence Date]

Contents of This Guide

1. How to Defend the Project — Strategy & Structure
2. The 30-Second Elevator Pitch
3. Suggested Presentation Flow (with timing)
4. Live Demo Script — Page by Page
5. System Architecture & Technology Choices
6. The Direct-Access (Random Indexing) Argument — your core concept
7. Page-by-Page Guide: What Each Page Does & How to Use It
8. Roles & Permission Matrix
9. Key Formulas & Calculations
10. Security Measures (be ready to explain)
11. Defence Question Bank — 40 likely questions with model answers
12. Evaluation Against Project Objectives
13. Limitations & Future Work
14. Presentation Tips & Common Pitfalls
15. Glossary of Key Terms

How to use this guide: Sections 1–4 prepare your delivery and demo. Sections 5–6 give you deep technical understanding. Sections 7–10 are reference material. Section 11 is the most important part to rehearse — read each question, close the guide, and answer out loud until it sounds natural.

1. How to Defend the Project — Strategy & Structure

1.1 The examiner's mindset

The panel is not trying to fail you. They want to confirm three things:

- **You understand the problem** you set out to solve and why it matters.
- **You actually built it** — it works, and you can show it working.
- **You understand your own code and decisions** — why you chose each technology, how the data flows, and what its limits are.

Every question they ask maps to one of those three. Answer accordingly: state the fact, then connect it back to the project's goal.

1.2 The formula for a great answer

POINT (answer directly) → PROOF (show/quote the system) → WHY (reasoning / trade-off)

Example: “Why did you use PostgreSQL?” → “A relational database gives me structured financial records with integrity (POINT). The schema stores users, pumps, shifts and readings with foreign keys and unique constraints (PROOF). I chose Postgres specifically because the system deploys on Vercel’s serverless platform, where a hosted Postgres keeps data persistent, and it supports the B-tree indexes that give my direct-access queries $O(\log n)$ speed (WHY).”

1.3 Structure of a typical defence (10–15 minutes + questions)

Time	Segment	What you do
0–2 min	Introduction	Title, your name, supervisor, and a one-sentence summary of the project.
2–4 min	Problem & Motivation	Manual bookkeeping failures; slow sequential systems; why SMEs and filling stations specifically.
4–6 min	Proposed Solution & Architecture	Direct-access database, web application, three-tier flow: browser → API → database.
6–10 min	Live Demonstration	Walk through the pages (Section 4 script). Let the system do the talking.
10–12 min	Results / Evaluation	Show reconciliation accuracy, variance detection, report outputs.
12–15 min	Conclusion & Future Work	Restate the contribution; list 2–3 realistic extensions.
remaining	Questions	Answer confidently using the question bank (Section 11).

1.4 Golden rules during the defence

- **Demo first, talk second.** Examiners love seeing it work. Keep narrative short during the demo.
- **Know the numbers.** Remember key facts: 4 pumps, 3 tanks, 3 fuel types, 3 roles, ~15 API endpoints, 6 database tables.
- **If you don't know, be honest.** Say *"I haven't tested that specifically, but based on how X is implemented, I would approach it by..."* Honesty plus reasoning scores better than bluffing.
- **Never overclaim.** The project does not do payroll, does not read pump telemetry, and is designed for a single station. Saying otherwise invites follow-up questions that fail you.
- **Have a backup plan** for the demo: production build running locally, screenshots saved, and the data re-seeded (one command: `npm run db:reset`).
- **Dress and speak like a professional.** Slow down, look at the panel, and pause after each point.

2. The 30-Second Elevator Pitch

Memorise this. It is the answer to the very first question in almost every defence: *“Summarise your project.”*

“My project is a web-based Financial Information System for small and medium enterprises, built for a retail filling station as a case study. Independent stations still manage daily fuel sales and cash using manual ledgers, which are slow, error-prone, and vulnerable to loss — and early computerised systems were limited by sequential storage, so retrieving any historical record took longer as data grew. I designed and implemented a system on a direct-access database architecture: every record is located by its hash-indexed key in constant time, and date-range queries use B-tree indexes. The system lets pump attendants record meter readings, lets supervisors reconcile expected pump revenue against cash and POS collections instantly, and lets managers monitor underground tank inventory to detect shrinkage. It runs as a Next.js application with a REST API and a PostgreSQL database, organised in a monorepo so the whole system deploys to Vercel as one project.”

(~35 seconds at a normal speaking pace — trim the last sentence if needed.)

3. Suggested Presentation Flow

Use this slide/page order. Each slide has one main idea and one visual.

Slide	Title	Content / Visual
1	Title slide	Project name, student, supervisor, date.
2	The Problem	Manual ledger weaknesses (human error, physical loss); sequential processing latency (reading 299 days to reach day 300).
3	Why SMEs / Why a filling station	High-volume, low-margin, cash-intensive; high risk of revenue leakage and tank shrinkage.
4	The Solution	Direct-access architecture + web application; diagram: Attendant → Browser → API → Database.
5	Objectives → Features map	Table: each objective from the proposal and the module that fulfils it (Section 12).
6–9	Demo	Login → Dashboard → Shift Sales → Inventory → Reports (use the script in Section 4).
10	Evaluation	Reconciliation accuracy; variance detection; reports for audit.
11	Architecture & Security	Monorepo layout, tech stack table, auth/security bullet points.
12	Conclusion & Future Work	Contribution + extensions (telemetry, multi-station, payroll).
13	Thank you / Questions	One line: “Thank you — I welcome your questions.”

Do not read the slides. Slides hold keywords; you speak the explanation. **Do not** rush the demo — the demo is your strongest evidence.

4. Live Demo Script — Page by Page

Practice this until it takes you 4–5 minutes. The seeded database contains 7 days of closed shifts and one open shift for today, so every screen has realistic data on first load.

Step 0 — Preparation (before the panel arrives)

- Start the app: `npm run dev` (or `npm run build && npm run start`).
- If the data was modified during rehearsal, reset it: `npm run db:reset` (this re-seeds automatically).
- Keep a terminal window visible so you can show the server logs running.

Step 1 — Login (20 seconds)

Say: *“The system uses role-based accounts. I’ll sign in as the manager.”*

- Open `/login`, enter `admin@udyking.com` / `admin123`.
- Point out: password is hashed in the database; the session is an `httpOnly` cookie, so it cannot be read by browser scripts.

Step 2 — Dashboard (45 seconds)

Say: *“The dashboard gives management an at-a-glance operational view.”*

- Stat cards: *“Today’s fuel sold in litres, today’s expected revenue, open-shift status, active pumps.”*
- 7-day revenue chart: *“This is generated from the database’s indexed records — no manual totalling.”*
- Open-shift card and Product-Loss Alerts: *“If any tank shows physical depletion above recorded sales, an alert appears here immediately.”*
- Recent shifts table with variances highlighted in red where cash did not match pump revenue.

Step 3 — Shift Sales (2 minutes — the heart of the project)

Say: *“This is the daily operational workflow. An attendant opens a shift and records pump meter readings; a supervisor later reconciles the cash.”*

- Because a shift is already open (seeded for today), the board shows one row per pump: Pump 1 and 2 (PMS at 11,150/L), Pump 3 (AGO at 11,300/L), Pump 4 (DPK at 11,250/L).
- Point at the live computation: change the closing reading on one pump and show litres-sold and expected revenue recomputing instantly (no manual multiplication).
- Press **Save Pump Readings**; show the confirmation message.
- Enter a cash and POS amount for one pump: show the variance column turning red when cash + POS is below expected revenue. *“The system does not accuse anyone — it flags the gap so the supervisor investigates before closing.”*
- Click **Close Shift & Reconcile**; the totals row shows expected, actual and variance, and the shift moves into history.
- Optionally open a new shift with the **Open Today’s Shift** button so the panel sees that flow too.

Step 4 — Shift History / Shift Detail (30 seconds)

- Click **View** on a closed shift: show the full breakdown per pump — opening/closing readings, litres, expected, cash, POS, variance, and who opened and closed it. *“Every figure is traceable to a person and a time — that is the audit trail.”*

Step 5 — Inventory (45 seconds)

Say: *“Inventory control is what protects the station from shrinkage — leaks, evaporation, or unrecorded discharge.”*

- Show the history table: opening level, litres received, pump sales, expected closing, actual closing, and the variance column (losses in red).
- Record a new check: pick the PMS tank, enter opening level, received litres and dip closing level. *“The system calculates expected closing as opening plus receipts minus what the pumps recorded, and compares it with the physical dip reading. A closing level below expected means product is missing.”*

Step 6 — Reports (45 seconds)

Say: *“Reports turn the raw records into management information for daily, weekly and monthly audits.”*

- Daily Sales tab: totals (litres, expected revenue, cash, POS, net variance) plus a per-day chart and table.
- Monthly Summary tab: revenue by month across the selected range.
- Inventory tab: volumetric report with net loss/surplus.
- Show the date-range picker: *“Any range queries against the database run on B-tree indexes — this is where the direct-access architecture removes the sequential bottleneck described in the problem statement.”*

Step 7 — Pumps & Tanks, Staff (30 seconds)

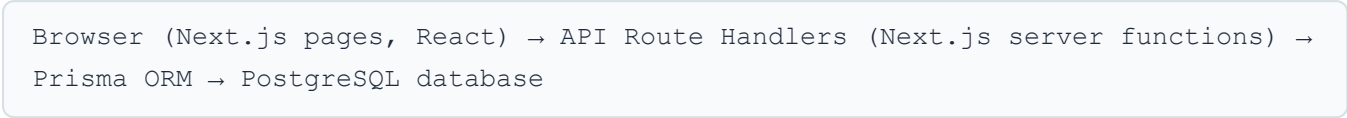
- Pumps & Tanks: show adding a pump, editing a price (price changes are snapshotted per reading, so history stays accurate), activating/deactivating.
- Staff: show creating an attendant account and changing a role. *“Access is role-based — an attendant cannot close a shift or see staff management.”*

Step 8 — Role demonstration (optional, 30 seconds)

- Sign out, sign in as `attendant@udyking.com` / `attendant123`.
- Show that the navigation bar no longer shows Pumps & Tanks or Staff, and that the reconciliation inputs are hidden. *“The interface adapts to the role, and the API enforces it too — even a crafted request is rejected with 403.”*

5. System Architecture & Technology Choices

5.1 Architecture overview



- **Frontend:** Next.js 15 App Router with React 19 server components. Pages render on the server with data fetched directly from the database; interactive forms are small client components that call the API.
- **API layer:** REST endpoints under `/api/*` (auth, shifts, readings, inventory, pumps, tanks, users, reports) — these are serverless functions on Vercel.
- **ORM:** Prisma — type-safe database access with parameterised queries (protection against SQL injection) and automatic migration files.
- **Database:** PostgreSQL — relational integrity, transactions, B-tree indexes, and persistent hosting on Vercel via Neon.

5.2 Monorepo layout (be ready to explain this)

Folder	Purpose
<code>apps/web</code>	The Next.js application: pages (<code>src/app</code>), API routes (<code>src/app/api</code>), components, middleware, auth libraries.
<code>packages/db</code>	Prisma schema, migrations, seed script, and the shared database client used by the web app.
<code>packages/shared</code>	Shared TypeScript types, constants and formatters (roles, fuel types, currency/litre formatting) reused across the app.
<code>vercel.json</code>	Tells Vercel to build <code>apps/web</code> and run database migrations on every deploy.

Why a monorepo? One repository, one install, one deployment. The frontend, API and database layer live together, shared code is imported directly (no version publishing), and the whole system deploys to Vercel as a single project — the user asked specifically for this so hosting is one click instead of managing separate servers.

5.3 Data model (six tables)

Table	Key fields	Relations
User	name, email (unique), passwordHash, role (MANAGER / SUPERVISOR / ATTENDANT)	opens and closes shifts; records inventory checks
Pump	name (unique), fuelType (PMS/AGO/DPK), pricePerLiter, active	one pump → many readings

Tank	name, fuelType (unique), capacityLitres	one tank → many inventory checks
Shift	date, status (OPEN/CLOSED), openedBy, closedBy, totals	one shift → many readings
PumpReading	shift, pump, opening/closing meter readings, litresSold, price snapshot, expectedRevenue, cashReceived, posReceived, variance	unique per (shift, pump)
InventoryCheck	date, tank, openingLevel, receivedLitres, salesLitres, expectedClosing, closingLevel, varianceLitres	unique per (date, tank)

Talk point: financial integrity is enforced in the schema itself — a pump cannot appear twice in one shift, one tank can be checked once per day, and a shift can only be closed by an authorised supervisor or manager.

5.4 Why these technologies? (quick answers)

Choice	Why
Next.js	One framework for pages, API routes, and server-side rendering; free hosting on Vercel; file-based routing keeps the codebase small.
React / TypeScript	Reusable components (charts, forms, tables) and compile-time type safety across the whole project.
PostgreSQL	Relational integrity for financial data, B-tree indexes for direct access, and persistence on Vercel's serverless platform (via Neon).
Prisma	Type-safe queries, parameterised SQL (SQL-injection safe), and versioned migrations stored in the repo.
Tailwind CSS	Fast, consistent, responsive UI without writing custom CSS files.
JWT + bcrypt	Stateless session token in an httpOnly cookie; bcrypt salted hashing for passwords.
npm workspaces (monorepo)	Single codebase, shared packages, single Vercel deployment.

6. The Direct-Access (Random Indexing) Argument

This is the intellectual core of the project, directly linked to your Statement of the Problem. Understand it deeply — expect at least one question about it.

6.1 The problem it solves

Manual ledgers and early automated systems store records sequentially: to retrieve a record you scan from the beginning until you find it. For a filling station generating hundreds of records per day, retrieving one month of history from a year of records means reading most of the year first. Retrieval time grows with the dataset (linear, $O(n)$).

6.2 How this system achieves direct access

- **Primary keys are hash-indexed identifiers.** Every record (shift, reading, check, user) gets a unique key. The database maps that key directly to the record's physical location — a point lookup returns the record in **$O(1)$ constant time** regardless of how many records exist. This is random access / direct addressing: no linear traversal, exactly the architecture your proposal describes.
- **Range queries use B-tree indexes.** Queries like “*all sales between date A and date B*” or “*all readings for pump 3*” are accelerated by B-tree indexes on date and foreign-key columns. A B-tree keeps keys sorted and self-balancing, so lookup and range scans run in **$O(\log n)$** — finding data at “day 300” costs the same as finding “day 3”.
- **The difference is observable in the reports module:** selecting any 30-day window returns totals instantly because the database never scans records outside the indexed range.

6.3 Ready answers about indexes and complexity

- **What is a B-tree?** A self-balancing tree data structure that stores sorted keys. Each node has many children, keeping the tree shallow; searches, inserts and deletes stay $O(\log n)$. It is the standard structure database engines use under their indexes. (This matches the B-tree definition in your Definition of Terms.)
- **Where exactly are the indexes in your schema?** Unique indexes on the primary keys of all six tables; a unique compound index on `(shiftId, pumpId)` for readings and `(date, tankId)` for inventory checks; additional indexes on `pumpId` and `tankId` foreign keys. Date columns are indexed implicitly through the compound keys and query patterns.
- **Is it really $O(1)$?** For primary-key lookups, yes — the database resolves the key to a location directly (hash or B-tree root descent). For range queries it is $O(\log n + k)$ where k is the number of returned rows — still logarithmic, never a full scan.
- **What would happen if you used the old sequential approach?** The reports query would read every stored record every time, and the time to answer would grow linearly with business age — the exact “compounding retrieval latency” your problem statement describes.

Key sentence to say: “Direct access means the system's response time is independent of how long

the station has been operating — the first day and the 1,000th day answer at the same speed.”

7. Page-by-Page Guide: What Each Page Does & How to Use It

7.1 Login Page (/login)

Access: Public (no session required) | **Roles:** All | **API:** `POST /api/auth/login`

What it does: Authenticates station staff by email and password. On success the server verifies the password hash (bcrypt), issues a signed JWT containing the user id and role, and stores it in an httpOnly, same-site session cookie (7-day expiry). The user is then redirected to the dashboard (or the page they originally requested).

How to use: Enter the account email and password, click Sign in. Invalid credentials show a generic “Invalid email or password” (no account enumeration). An already-authenticated visitor is redirected away automatically.

7.2 Dashboard (/dashboard)

Access: All roles | **Data:** read from all six tables

What it does: The management overview. Four stat cards (today’s litres sold, today’s expected revenue, open-shift status, active pump count), a 7-day revenue bar chart, the current open shift’s status, the five most recent product-loss alerts (negative tank variances), and the five most recent shifts with their variance highlighted.

How to use: Read-only. Use the buttons to jump into Shift Sales or continue the open shift. Alerts in red deserve investigation first thing each morning.

7.3 Shift Sales (/sales) — the core workflow

Access: All roles (closing restricted to Supervisor/Manager) | **API:** POST /api/shifts , POST /api/shifts/[id]/readings , POST /api/shifts/[id]/close

What it does: Manages the daily shift lifecycle.

1. **No open shift:** shows an “Open Today’s Shift” card. Only one shift can be open at a time (the API rejects a second one — server-side guard).
2. **Open shift board:** one row per active pump. The attendant enters the pump’s opening and closing meter readings; litres sold and expected revenue compute live in the browser and are re-validated on the server. Opening readings are prefilled from the previous day’s closing.
3. **Reconciliation (Supervisor/Manager only):** cash and POS collected per pump are entered; the variance per pump and the shift totals update live. “Close Shift & Reconcile” locks the shift, records who closed it and when, and moves it to history.
4. **History table:** the ten most recent closed shifts with expected/actual/variance and a link to the full detail page.

How to use (attendant): open shift → read the pump meters at opening and at close → type both readings → Save. **(supervisor):** count the cash, collect POS receipts → enter per pump → check the variance → Close.

7.4 Shift Detail (/sales/[id])

Access: All roles | **API:** GET /api/shifts/[id]

What it does: Read-only, printable breakdown of one shift: who opened and closed it and when, every pump’s readings, litres, expected revenue, cash, POS and variance, with totals. This is the audit record for any dispute or audit request.

How to use: Open from Shift Sales history (View) or the dashboard’s recent shifts. Back button returns to Shift Sales.

7.5 Inventory & Storage Control (/inventory)

Access: All roles (recording restricted to Supervisor/Manager) | **API:** GET/POST /api/inventory

What it does: Volumetric reconciliation of underground tanks against recorded pump sales. The form takes the date, tank, dip opening level, litres received, and dip closing level. The system computes pump sales for that fuel on that date, then expected closing = opening + received – sales, and flags the difference as a loss (closing below expected) or surplus. History shows every check with OK / loss / surplus badges.

How to use: Each day (or per shift), dip each tank, enter the levels, save. Watch the loss column: consistent negative variance on one tank points to a leak, evaporation, or unrecorded discharge that needs physical investigation.

7.6 Reports (/reports)

Access: All roles | **API:** `GET /api/reports/sales`, `GET /api/reports/inventory`

What it does: Three report tabs over any date range:

- **Daily Sales:** total litres, expected revenue, cash, POS and net variance; revenue-per-day chart and a per-day table.
- **Monthly Summary:** the same aggregates grouped by month — the basis for monthly financial audits.
- **Inventory:** all tank checks in the range with net loss/surplus totals.

How to use: Pick From/To dates, click Run Report. Defaults to the last 30 days.

7.7 Pumps & Tanks (/pumps) — Manager only

Access: Manager | **API:** `/api/pumps`, `/api/tanks`

What it does: Configures the station's physical assets. Pumps: name, fuel type, price per litre (click the price to edit), activate/deactivate, delete (only if it has no sales history). Tanks: name, fuel type (one tank per fuel), capacity, delete (only if unused).

How to use: Add a new pump before it starts operating; update the price when the station price changes — the price is snapshotted onto each reading, so historical revenue is never rewritten. Deactivate, don't delete, a pump with history — this preserves the audit trail.

7.8 Staff (/users) — Manager only

Access: Manager | **API:** `/api/users`, `/api/users/[id]`

What it does: Creates staff accounts (name, email, password, role), changes roles, resets passwords, deletes accounts that have no record history. The manager cannot change their own role or delete their own account (self-protection).

How to use: Add each staff member with the role matching their duty; when an attendant is promoted to supervisor, change the role so reconciliation permission moves with them.

8. Roles & Permission Matrix

Capability	Attendant	Supervisor	Manager
View dashboard, sales, inventory, reports	✓	✓	✓
Open a shift	✓	✓	✓
Enter / update pump readings	✓	✓	✓
Enter cash/POS and close (reconcile) a shift	—	✓	✓
Record inventory checks	—	✓	✓
Manage pumps, tanks and prices	—	—	✓
Manage staff accounts and roles	—	—	✓

Enforcement points (say this in the defence): permissions are enforced in three layers — (1) the UI hides restricted controls per role, (2) every API route calls `requireUser([...])` and returns 403 for unauthorised roles, and (3) the navigation middleware verifies the session on every protected request. Hiding a button is never the security boundary — the server is.

9. Key Formulas & Calculations

$\text{Litres sold} = \text{Closing meter reading} - \text{Opening meter reading}$

$\text{Expected revenue} = \text{Litres sold} \times \text{Price per litre (snapshot at time of reading)}$

$\text{Cash variance (per pump)} = \text{Expected revenue} - (\text{Cash collected} + \text{POS collected})$

Shift totals: $\text{expectedTotal} = \Sigma \text{ expected}; \text{actualTotal} = \Sigma (\text{cash} + \text{POS});$
 $\text{varianceTotal} = \Sigma \text{ variance}$

Inventory: $\text{Expected closing level} = \text{Opening level} + \text{Litres received} - \text{Pump sales}$
(same fuel, same date)

$\text{Inventory variance} = \text{Actual closing level} - \text{Expected closing level}$ (negative = loss/shrinkage; positive = surplus)

Validation rules the panel may probe:

- Closing reading < opening reading is rejected (400 error) — prevents nonsense negative litres.
- Negative cash or POS is rejected.
- Only one open shift at a time, enforced server-side.
- Reconciliation is only possible after readings are saved, and only while the shift is OPEN.
- Closed shifts are immutable — no edits after closing (audit integrity).
- One inventory check per tank per day (upsert), so re-entering a day corrects rather than duplicates.

10. Security Measures (be ready to explain each)

Threat	Countermeasure in this system
Password theft	bcrypt salted hashing (cost factor 10); plaintext passwords never stored; passwords never returned by the API.
Session hijacking via XSS	Session token stored in an httpOnly cookie — JavaScript cannot read it, so even a successful script injection cannot steal the session.
Token forgery	JWT signed with a server-side secret (HS256, <code>JWT_SECRET</code>); forged or expired tokens are rejected.
CSRF	SameSite=Lax cookie, JSON-only APIs, and no cookie-authenticated state-changing requests from foreign origins.
SQL injection	All queries go through Prisma's parameterised query engine — user input is never concatenated into SQL.
Stored/reflected XSS	React escapes all rendered values by default; user-entered text is treated as data, never markup.
Privilege escalation	Role checks on every API route (<code>requireUser(roles)</code>); the UI hiding controls is cosmetic, the server rejects.
Account enumeration	Login failures return a generic message; registration of duplicate emails is the only user-exposed difference, which is unavoidable and standard.
Self-lockout / admin abuse	A manager cannot delete their own account or change their own role.
Input errors / bad data	Zod schema validation on every API body and query before any database write.

11. Defence Question Bank (40 questions with model answers)

11.1 Opening / general questions

Q1. Summarise your project in 2–3 sentences. Opening

Use the 30-second pitch from Section 2, trimmed: problem (manual ledgers and slow sequential systems), solution (direct-access financial information system for a filling station), result (role-based web app: sales logging, cash reconciliation, tank inventory control, reports).

Q2. Why did you choose a filling station as your case study? Motivation

Filling stations are high-volume, low-margin, cash-intensive businesses where small leakages matter. Three pressures make them a strong case: daily sales on multiple pumps, cash-heavy payment mix where diversion is easy, and underground tanks where product can disappear silently through leaks or theft. Those are exactly the problems my three modules address — sales logging, reconciliation, and volumetric inventory control. And the motivation is personal: I want to build technology that directly supports SME owners with a scalable, affordable system.

Q3. How is this different from using Excel or a simple spreadsheet? Motivation

Four differences: (1) integrity — the database schema enforces rules (one open shift, unique readings per pump, non-negative values) that a spreadsheet cannot; (2) access control — role-based permissions stop an attendant from altering reconciliation figures; (3) auditability — every record knows who created it and when; (4) retrieval performance — indexed queries return any historical range instantly, while a spreadsheet over thousands of rows slows down and corrupts easily. Excel is a single-user document; this is a multi-user information system.

Q4. Who are the users of your system? Scope

On-site operational staff only: pump attendants (open shifts, enter meter readings), sales supervisors (reconciliation, inventory checks), and the station manager (configuration, staff accounts, reports). Per the project scope, it is deliberately local to one outlet, not a multi-station network.

Q5. What problem does the system solve in plain terms? Motivation

Cash and fuel disappearing without the owner knowing, and history being slow to retrieve. Manual tallies make it hard to spot a missing naira; tank dips at the end of the month are the only way losses were noticed. This system flags discrepancies daily — expected revenue versus cash collected, and physical tank depletion versus recorded sales — while the owner can pull any day's figures instantly.

Q6. What scope did you deliberately exclude? Scope

Payroll and staff remuneration, broad enterprise accounting (balance sheets, tax computations, depreciation), and automated hardware telemetry. Data entry is manual through the web interface, as stated in the delimitations. Excluding them kept the project focused and achievable while still solving the core cash-and-inventory problem.

11.2 Architecture and database questions

Q7. Explain your system architecture. Architecture

Three tiers in one deployable unit: a Next.js frontend (React server components rendering pages), a REST API layer (Next.js route handlers under /api), and PostgreSQL accessed through the Prisma ORM. In the monorepo, packages/db owns the schema and migrations, packages/shared owns shared types, and apps/web is the application. On Vercel, all of this runs as serverless functions connected to a hosted Postgres database, so there is no separate backend server to manage.

Q8. Why Next.js instead of just React? Architecture

Next.js gives me three things in one framework: server-side rendering (pages can read the database directly, so the dashboard loads with data, not after a client fetch), API route handlers (so the backend lives in the same codebase and deploys together), and a file-based router that keeps the project small. It also integrates natively with Vercel hosting, which was a requirement of the project.

Q9. Why a monorepo? What does each part do? Architecture

One repository containing the web app, the database package and the shared package. The database package centralises the Prisma schema and migrations so one migration history exists; the shared package removes duplicated types and formatters; the web app imports both directly via npm workspaces. The result is a single project on Vercel — one build, one domain — which the project brief required.

Q10. Why PostgreSQL and not MySQL or SQLite? Database

Three reasons: relational integrity for financial data (foreign keys, unique constraints, transactions), B-tree indexing for the direct-access requirement, and deployment — Vercel's serverless filesystem is ephemeral, so the database must be hosted; Postgres on Neon is the natural choice. The proposal mentioned MySQL; Prisma supports both, so the schema could switch providers with a configuration change, but Postgres was chosen for the hosting story.

Q11. Explain how your system achieves direct access / random indexing. Core concept

Every record is stored with a unique primary key that the database maps directly to a storage location — a point lookup is $O(1)$ regardless of dataset size. For range queries, B-tree indexes on dates and foreign keys give $O(\log n)$ access. So retrieving sales from 300 days into operation costs the same as retrieving yesterday's — the sequential bottleneck from the problem statement is eliminated. The reports module demonstrates this: any date window returns totals instantly.

Q12. What is a B-tree and where is it used in your system? Core concept

A self-balancing sorted tree where each node holds many keys, keeping the tree shallow, so search, insert and delete are $O(\log n)$. The database engine builds B-tree indexes behind my unique keys and compound indexes (shiftId + pumpId for readings, date + tankId for inventory checks). Every query that filters on those columns walks the index instead of scanning the table.

Q13. Draw/explain your database schema. Database

Six tables: User, Pump, Tank, Shift, PumpReading, InventoryCheck. A Shift has many PumpReadings (one per pump, enforced by a unique constraint). A reading stores meter readings, litres, the price snapshot and expected revenue, plus cash, POS and variance filled at reconciliation. InventoryCheck is unique per tank per day. Users relate to shifts they opened/closed and checks they recorded — the audit trail.

Q14. How do the pages and the API communicate? Architecture

Most pages are server components that query the database directly through Prisma on the server — the page HTML ships with data already. Interactive actions (saving readings, closing a shift, creating a user) are handled by small client components that POST JSON to the API routes; the server validates with Zod, enforces roles, writes with Prisma, and returns JSON; the page then refreshes. Reports are client-side and fetch from the report APIs with date parameters.

Q15. What is an API route handler? Architecture

A server function defined by a file in the app/api directory that receives an HTTP request and returns an HTTP response — e.g. POST /api/shifts/[id]/close closes a shift. On Vercel each handler becomes a serverless function. This keeps the whole backend in the same TypeScript codebase as the frontend.

Q16. How do you handle concurrency, e.g. two attendants opening shifts at once? Architecture

The invariant “at most one OPEN shift” is enforced in the database layer: the API checks for an existing open shift before creating, and the business rule is also encoded in the application logic. For typical station scale (one or two staff per shift) this is sufficient; if needed it could be hardened with a partial unique index on status = OPEN.

Q17. Why store a price snapshot on each reading instead of just referencing the pump's current price? Design decision

Because fuel prices change. If the manager updates the price today, historical readings must still show the revenue actually expected on that day. By copying pricePerLiter onto each reading at save time, history is immutable and reports stay correct — this is a standard accounting principle: store the transaction facts as they were.

11.3 Security questions

Q18. How does authentication work? Security

On login, the server compares the submitted password against the bcrypt hash in the database. On success it signs a JWT containing the user id and role, with a 7-day expiry, and sets it in an httpOnly, SameSite=Lax cookie. Every subsequent request verifies the token signature and expiry; the middleware guards protected pages and API routes, and route handlers re-verify the role. Logout deletes the cookie.

Q19. Why is the session in a cookie and not in localStorage? Security

localStorage is readable by any JavaScript on the page, so a single XSS vulnerability would leak the session token. An httpOnly cookie is invisible to JavaScript — it is sent automatically with requests and can only be set by the server. That is the standard, safer choice for session material.

Q20. How do you prevent SQL injection? Security

All database access goes through the Prisma ORM, which uses parameterised queries — user input is bound as parameters, never concatenated into SQL strings. Combined with Zod validation of every API input, there is no path for an attacker's input to alter the query structure.

Q21. How is role-based access enforced? Security

In three layers: the UI conditionally renders privileged controls; the navigation middleware verifies the session on every protected route; and every API handler calls requireUser(roles) before doing anything — unauthorised roles receive 403. Server-side enforcement is the real boundary; the UI is only convenience. I demonstrated this: an attendant's close-shift request is rejected with 403 even if crafted directly.

Q22. How are passwords stored? Security

With bcrypt at cost factor 10 — a salted, computationally expensive one-way hash. The plaintext never touches the database or any response; even the API only ever returns the user's id, name, email and role.

Q23. What about XSS and CSRF? Security

For XSS: React escapes all rendered values by default, so stored names or messages are treated as text, not markup. For CSRF: the session cookie is SameSite=Lax, and state-changing calls are JSON-only APIs, which a cross-site form cannot send; combined with the JSON content-type requirement, a cross-origin attacker cannot forge state changes.

11.4 Functional / domain questions

Q24. Walk me through the daily sales workflow. Domain

An attendant opens the shift; the system stamps who and when. The attendant records each pump's opening and closing meter readings; litres and expected revenue compute automatically at the pump's price snapshot. At close, a supervisor enters cash and POS per pump; the system flags any variance versus expected revenue. Closing locks the shift with totals and the auditor's identity, and the record flows into history and reports.

Q25. How do you detect theft or leakage of fuel? Domain

Through volumetric reconciliation. For each tank and day: expected closing = opening dip + litres received – litres the pumps recorded. If the physical dip closing is below expected closing, product is missing — either a leak, evaporation, or unrecorded discharge. Negative variances are highlighted in red on the inventory page and surfaced as alerts on the dashboard so the owner investigates the same day instead of discovering it at month-end.

Q26. What happens when cash collected does not match expected revenue? Domain

The variance is computed per pump and per shift and shown immediately in red. The system does not judge — it flags, so the supervisor can investigate (e.g. an unattended sale, a pricing change, a faulty pump) before closing. The variance is stored with the shift for the audit trail and appears in reports.

Q27. What validation prevents bad data? Domain

Server-side: closing readings cannot be less than opening readings, cash/POS cannot be negative, dates must be valid, one open shift at a time, reconciliation only after readings exist, one inventory check per tank per day. Every input is validated with Zod before reaching the database, so validation can never be bypassed by editing the browser.

Q28. Can an attendant fix a wrong reading? Domain

Yes — while the shift is open, readings can be re-saved (the record is updated in place, unique per pump). Once the supervisor closes the shift, it is locked and no edits are allowed, preserving the integrity of the audit trail. This mirrors real practice: correct before sign-off, not after.

Q29. Why Float for money instead of Decimal? Design decision

Honest answer: simplicity. The values are rounded to two decimals everywhere, and the magnitudes at station scale (up to tens of millions of naira) are well within float precision for the rounding used. I noted in the design that Decimal is the stricter accounting choice and would be the improvement if precision at the kobo level ever mattered — Prisma supports Decimal natively, so the change is localised.

Q30. What happens if the system is closed mid-shift (e.g. power cut)? Domain

Nothing is lost: readings are saved to the database the moment “Save” is clicked, and the shift remains OPEN until explicitly closed. On reopening the app, the open shift resumes with its saved readings. Since it is a web application, the attendant could even continue from another device on the same network.

Q31. How are dates and time zones handled? Design decision

All date values are normalised to local midnight at the point of entry (a start-of-day helper) before storage, so a “shift for 2 August” is always stored as 2 August, and day/month grouping is consistent across the app regardless of the server time zone. Time-of-day fields (openedAt, closedAt) are stored as timestamps.

11.5 Business impact and evaluation questions

Q32. How does this benefit the station owner financially? Impact

Three ways: detection of revenue leakage (variance flagging), detection of product loss (volumetric reconciliation), and structured records that make the business bankable — an auditable digital ledger supports loan applications and tax compliance, which the proposal lists as a national-economy benefit. Faster daily reconciliation also frees staff time.

Q33. How did you evaluate the system against your objectives? Evaluation

Objective 1 (direct-access database + friendly UI): delivered — the schema uses indexed primary/foreign keys and the interface is role-based; retrieval is instant in reports. Objective 2 (full-stack implementation): delivered — Next.js + TypeScript + Prisma + PostgreSQL in a monorepo. Objective 3 (evaluate accuracy and usability): I seeded realistic transaction data and verified that every computed figure (litres, revenue, variance, inventory loss) matches manual recalculation, and that reports reconcile exactly with the stored shifts.

Q34. How scalable is this system? Impact

The architecture scales in three directions: data volume (indexed queries stay logarithmic as years of history accumulate), users (serverless functions scale automatically), and stations (the schema already separates pumps, tanks and users; adding a stationId column and an extra foreign key would extend it to multi-branch use). That said, the current scope is deliberately one outlet.

Q35. What does it cost to run? Impact

Near zero at this scale: Vercel’s hobby tier hosts the app, and a free-tier hosted Postgres (Neon) holds the data. For an SME this is effectively a low-cost solution — matching the proposal’s requirement that SMEs need affordable technology.

Q36. How did you test the system? Evaluation

Four levels: (1) schema-level — unique constraints and enums; (2) API-level — I exercised every endpoint with curl, including negative cases (401 unauthenticated, 403 wrong role, 400 bad readings); (3) end-to-end — a full day's cycle: open shift, save readings, reconcile, close, verify shift totals and the report output; (4) seed-based — 8 days of realistic data verified against manual arithmetic.

Q37. What are the limitations of the current system? Evaluation

Manual data entry only (no pump telemetry or tank gauge integration), single-station scope, no payroll or full accounting modules, and float arithmetic rather than exact decimal for money. Each is documented; the telemetry and multi-station items are future work, and I've been explicit that the system targets operational cash-and-inventory control, not full enterprise accounting.

11.6 Future work and closing questions

Q38. What would you add next if you continued the project? Future

Priority order: (1) automated tank telemetry and pump data to remove manual entry; (2) multi-station support with consolidated owner reporting; (3) CSV/PDF export of reports for auditors; (4) payroll and credit-customer ledger modules to broaden the accounting scope; (5) an offline-first mobile app or PWA so attendants can work during network outages; (6) shift-transfer locking so only the attendant on duty can edit their readings.

Q39. What did you personally learn from this project? Closing

End-to-end software engineering: translating a real business problem into a data model, designing an API, building a UI for non-technical users, and hardening it with validation and role-based security. I also learned that architecture decisions (direct access, hosting, monorepo) are answers to real constraints, and that honest scoping — deciding what not to build — is as important as deciding what to build.

Q40. If you had to start over, what would you do differently? Closing

I would prototype the data model earlier and validate it against the real station's daily forms before writing the UI, and I would add automated tests from day one rather than after the fact. I would also use Decimal for monetary columns from the start to be strictly accounting-correct, and design the station/tank relationships to allow multi-branch growth from the beginning.

Stretch questions (harder, for preparation): "What happens to variance if a pump gives free litres due to a fault?" (the variance becomes positive on that pump — cash collected exceeds expected; the flag prompts a pump calibration check). "How would you prevent an attendant from creating a fake zero-reading shift?" (role separation — only supervisors close; a supervisor would still need collusion; audit trail shows who opened). "What if two shifts were opened on different dates by mistake?" (open-shift guard is global, so only one can exist; the wrong one must be closed with a zeroed reconciliation — documented process gap I would address with a 'discard shift' permission for managers). "How would you handle the database filling up?" (partitioning by date or archiving closed shifts older than N years).

12. Evaluation Against Project Objectives

Map your proposal's three objectives directly — write these on your evaluation slide:

Objective (from proposal)	How it was fulfilled	Evidence
1. Design a direct-access database structure and user-friendly web interface for entering fuel sales and monitoring inventory.	Six-table relational schema with primary-key and B-tree indexes; role-based web interface with dedicated Sales and Inventory modules.	Schema file (<code>schema.prisma</code>); Shift Sales and Inventory pages; instant range queries.
2. Implement a functional web-based system using a full-stack approach (HTML/CSS, JavaScript, server-side language, relational database).	Next.js (React/TypeScript/Tailwind) frontend, Next.js API handlers as the server side, Prisma ORM, PostgreSQL database — in a monorepo.	Running application; 14 API routes; migration history; deployed architecture on Vercel.
3. Assess operational efficiency, data accuracy and user-friendliness using simulated or actual transaction data.	Seeded 8 days of realistic transactions; verified every computed figure against manual arithmetic; exercised role restrictions and error paths.	Reconciliation totals match manual recalculation; variance and loss detection verified; role matrix tests (401/403/400 cases).

13. Limitations & Future Work

13.1 Honest limitations (say them before the panel asks)

- Data entry is manual — no connection to pump or tank-gauge hardware.
- Single-outlet scope (per the delimitations).
- No payroll, tax computation, balance sheet or full external audit module.
- Money stored as Float (rounded), not exact Decimal.
- No automated test suite (validated manually and via API scripts).
- No offline mode — requires network connectivity.

13.2 Future work (realistic, prioritised)

1. Hardware telemetry integration (digital tank gauges, pump consoles) to remove manual entry.
2. Multi-station support with consolidated reporting for a chain of outlets.
3. CSV / PDF report export for auditors and banks.
4. Credit-sales (customer ledger) and payroll modules.
5. Offline-first mobile PWA for attendants.
6. Automated tests (unit + end-to-end) and Decimal monetary columns.

14. Presentation Tips & Common Pitfalls

Do

- Rehearse the demo at least three times, including the failure path (re-seed if data was modified).
- Open with one sentence that states the problem and the deliverable.
- Use the live system as your slides for the demo portion.
- Connect every technology answer back to a project requirement.
- Pause after each answer; invite follow-up.
- Keep a printed copy of this guide and the README with you.

Don't

- Don't claim features that don't exist (no telemetry, no payroll, no multi-station).
- Don't read slides or code line-by-line.
- Don't say "I just copied the code" — be able to explain any function the panel points at.
- Don't memorise answers word-for-word — memorise the structure (point, proof, why).
- Don't leave the demo unprepared: browser crashed → reopen; database reset → one command; both covered in Section 4.

Emergency cheat sheet (30 seconds before the defence)

- 4 pumps (2 PMS, 1 AGO, 1 DPK) · 3 tanks · 3 roles · 6 tables · 14 API routes.
- Core formula: variance = expected revenue – (cash + POS); inventory loss = actual dip – expected dip.
- Core claim: $O(1)$ point lookups, $O(\log n)$ range queries — no sequential scanning.
- Stack: Next.js + TypeScript + Tailwind + Prisma + PostgreSQL, monorepo, one Vercel deployment.
- Security: bcrypt, httpOnly JWT cookie, role checks server-side, parameterised queries, Zod validation.

15. Glossary of Key Terms

Term	Meaning (in your own words)
Financial Information System (FIS)	A software system that collects, processes, stores and retrieves financial data to support an organisation's operations and decision-making.
Direct access / random indexing	Retrieving any record directly from its stored location (via its key) without reading preceding records.
Sequential record processing	Reading records in storage order from the start; retrieval time grows with dataset size.
Constant-time complexity ($O(1)$)	An operation whose time does not change as the dataset grows — e.g. looking up a record by primary key.
B-tree indexing	A self-balancing tree that keeps sorted keys; searches and range scans run in $O(\log n)$.
Volumetric inventory reconciliation	Comparing recorded pump sales with physical tank levels to detect losses.
ORM (Object-Relational Mapper)	A layer that lets code work with database records as typed objects (Prisma).
API route handler	A server function that responds to an HTTP request path (e.g. <code>POST /api/shifts</code>).
Serverless function	Backend code that runs on demand on the hosting platform and scales automatically (Vercel).
JWT (JSON Web Token)	A signed, time-limited token carrying the user's identity and role.
bcrypt	A salted, computationally expensive password-hashing algorithm.
Role-based access control (RBAC)	Restricting actions based on a user's assigned role (Manager/Supervisor/Attendant).
Schema validation (Zod)	Declarative runtime validation of API inputs before they touch the database.
Migration	A versioned SQL script that evolves the database schema safely (tracked in the repo).
Monorepo	A single repository containing multiple packages/apps that share code and tooling.
Seed data	Realistic sample records inserted to make the system demonstrable and testable.